En este paso, hemos implementado varias funciones, en su mayoría sencillas, donde aquellas que siguen una estructura basada en expresiones regulares son muy similares entre sí.

A modo de resumen, creamos las expresiones regulares probando y analizando los ejemplos proporcionados por el dataset, así como algunas referencias externas como *Regex101*. Para extraer la información solicitada en cada caso, utilizamos el método *findall*, que genera automáticamente una lista con todas las coincidencias de la expresión regular en cada post del dataset. Es importante destacar que esta función se aplicaría a los posts de la columna post y no a la columna “clean_post”, ya que en esta última los posts ya han sido normalizados, lo que impide encontrar algunas de las informaciones que necesitamos extraer. Además, hemos complementado cada función con explicaciones en forma de comentarios en los códigos. Esta lógica nos ha servido para los primeros tres apartados, que trataban de extraer subreddits, URLs y números de teléfono; mientras que para los dos últimos casos, los razonamientos han sido diferentes.

Es relevante mencionar que, en la función destinada a extraer números de teléfono, estos pueden presentarse en diversos formatos dependiendo de la región, país o continente de origen. Por ello, creamos una expresión regular algo extensa para abarcar los formatos más comunes, los cuales observamos tanto en el dataset como en una búsqueda general. Estos formatos incluyen números de nueve dígitos con un posible “+” al principio, números separados por delimitadores estándar como guiones, puntos, barras, entre otros, y uno donde la región está indicada entre paréntesis.

Primera Hipótesis

En cuanto a la función para extraer fechas, nos basamos en el razonamiento de que, debido a la diversidad de formatos de fecha, ¿cómo podemos abarcar prácticamente todos?. La solución fue intentar convertir ese string de la posible a fecha a un tipo *datetime*, de forma que si esa conversión no genera un error, significa que ese string es susceptible de ser una fecha, por lo que lo almacenamos en una lista de fechas válidas. Finalmente, utilizando el formato "%d-%m-%Y" con *date.strftime*, conseguimos extraer las fechas en el formato día-mes-año, que es uno de los más comunes y fáciles de interpretar para nosotros.

Segunda hipótesis

Tras una búsqueda, encontramos que una muy buena forma para obtener resultados interesantes, es utilizando de nuevo uno de los módulos de la librería de *spaCy*. La idea base sobre esta nueva implementación, es aprovechar las herramientas para extraer entidades, y en este caso, en concreto quedarnos con aquellas que sean de tipo *Fecha* o *Tiempo*. A continuación, añadimos una explicación sobre el código en cuestión:

- *Extracción y Análisis de Entidades:*
    - Se itera sobre las entidades reconocidas en el objeto “doc”.
    - Si la entidad es una fecha (“DATE”) o una hora (“TIME"), se utiliza “dateparser” para analizar y convertir el texto de la entidad en un objeto de fecha.

- *Explicación Adicional:*
    - “doc.ents”: Contiene las entidades reconocidas en el texto, como fechas, horas, eventos, personas, organizaciones, etc.
    - “ent.label_”: Proporciona la etiqueta de la entidad, que indica el tipo de entidad (por ejemplo, “DATE”, “TIME”, “EVENT").
    - “dateparser.parse”: Convierte el texto de la entidad en un objeto de fecha, utilizando configuraciones específicas como el orden de la fecha. Con esto conseguimos lo que antes con el comando *strftime*, pero así probamos otras formas.

Extracción de código

Las ideas que hemos tenido para crear la última función han sido diversas. En primer lugar, lo planteamos utilizando únicamente expresiones regulares, pero no nos terminó de convencer, ya que resulta casi imposible abarcar todos los posibles formatos de código. Finalmente, hemos optado por utilizar una API, apoyándonos en herramientas como *GPT 4o*, porque creemos que la mejor opción a largo plazo sería entrenar un modelo con datos etiquetados que permitan distinguir si un string contiene código o no, lo que nos permitiría realizar predicciones precisas.

Dado que nuestro dataset es enorme y sería muy costoso obtener los datos de forma manual, hemos decidido aprovechar los datos disponibles en la web que ya están en el formato que nos interesa. Posteriormente, hemos seguido el mismo enfoque que en otros ejercicios y hemos utilizado un clasificador Random Forest como modelo para realizar las predicciones.

Una vez implementado este proceso, la idea es determinar si un string es código y, en caso afirmativo, almacenarlo en una lista, por ejemplo.

Explicación de la función final

Antes de implementar la función, intentamos recopilar datos de varias APIs, utilizando herramientas de Inteligencia Artificial, ya que no teníamos experiencia previa trabajando con APIs. Sin embargo, optamos por este enfoque porque era nuestra idea inicial para obtener datos de entrenamiento, que al final resultaron ser la principal fuente de información que necesitábamos. El resto del proceso, que incluye la implementación, el entrenamiento y la función final, ha sido desarrollado exclusivamente por nosotros.

Al obtener diversos conjuntos de datos etiquetados, los concatenamos en un único DataFrame, el cúal utilizamos para entrenar nuestro modelo. Tras el entrenamiento, obtuvimos muy buenos resultados, con un F1 Score superior a 0.97, lo que indica que el modelo parece ser fiable. Luego, combinamos varias expresiones regulares para extraer fragmentos relacionados con HTML y una función para identificar fragmentos de código de programación. La explicación de estas funciones se encuentra acompañada de comentarios explicativos en el código.

Cuando probamos estas funciones, obtenemos buenos resultados en general. Sin embargo, en el caso de los fragmentos de código, observamos que en ocasiones también se devolvían fragmentos que no correspondían estrictamente a código. Para abordar este problema, creamos una función auxiliar que analiza si la línea de texto coincide con alguno de los patrones definidos por tres expresiones regulares, las cuales hemos diseñado para identificar algunas de las palabras clave y caracteres más utilizados en Python.

Cabe señalar que estas expresiones regulares podrían ser más exhaustivas, pero también se volverían demasiado específicas. Esto podría generar falsos positivos, ya que en textos no relacionados con código, la falta de alguna palabra o expresión clave de Python podría hacer que se detectara erróneamente como código. Por esta razón, hemos adoptado un enfoque más conservador, limitando un poco el dominio de búsqueda para equilibrar la precisión y la flexibilidad del sistema.